CS 634
Data Mining
Final Project--Data Mining Models for Binary Classification: Determining Credit Card Defaults

Zoe Sedlacek

Table of Contents

References

Dataset used -- https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients

Sklearn -- https://scikit-learn.org/stable/index.html

Tensorflow -- https://www.tensorflow.org/

Pandas -- https://pandas.pydata.org/

Numpy -- https://numpy.org/

Project Description

This project implements three data mining classification algorithms--Random Forest, Decision Tree, and LSTM--to determine which credit card customers are likely to default on their payments. Each algorithm is analyzed over 10 folds using K-Fold cross validation and compared to the other algorithms to decide which is best suited to this problem. The submission includes the python code, in both python script and Jupyter Notebook format, the dataset, and this report document. The results include the metrics for each algorithm by individual fold and averaged over all folds as well as the time taken by each algorithm.

Summary

The coding language for this project is Python, written and tested in version 3.12. Non standard dependencies include sklearn, pandas, numpy, and tensorflow keras. Ensure that all requirements are installed and up to date on your system before running the program. All dependencies can be installed using “pip install ‘dependency name’”, and the program can be executed by running the executable python script in your file explorer or command line. If you have trouble or wish to know more about these dependencies, links are provided in the References section.

The dataset used for this project is Default of Credit Card Clients, created by I-Cheng Yeh. It can be found at the link in the references section. This dataset includes 23 features such as age, bill amounts, and payment history, and a binary label of negative for does not default and positive for does default. It includes 30000 rows of data from Taiwanese credit card clients, with 78% belonging to the negative class and 22% belonging to the positive class.

The models used in this project are Random Forest, Decision Tree, and LSTM. Random Forest creates a number of decision trees, also known as estimators, each using a subset of the training set to create the tree. The final classification is determined by a quorum of trees. Decision Tree only uses one decision tree, making it faster and simpler than Random Forest

but overall less powerful. Long Short Term Memory is a type of recurrent neural network composed of a memory cell and gates which let it remember and forget information over time. It can take a long time to train, but it has the ability to find complex dependencies which can improve its performance compared to other classifiers.

The metrics used to determine model performance are TN(the amount of negative samples correctly predicted as negative), FP(The amount of negative samples falsely predicted as positive), TP(the amount of positive samples correctly predicted as positive), and FN(The amount of positive samples falsely predicted as negative). From these, additional derived metrics such as recall, precision, F1 measure, accuracy, and error rate are calculated. The formulas and meaning of each derived measure are shown below.

Recall(r)	tp / (tp + fn)	The ratio of correct positive predictions to all positive samples
TNR	tn / (tn + fp)	The ratio of correct negative predictions to all negative samples
FPR	fp / (tn + fp)	The ratio of false positive predictions to all negative samples
FNR	fn / (tp + fn)	The ratio of false negative predictions to all positive samples
Precision(p)	tp / (tp + fp)	The ratio of correct positive predictions to all predicted positive samples
F1 measure(f1)	2 * (p * r)/(p + r)	Harmonic mean of precision and recall

Accuracy	(tp + tn) / (tp + fp + fn + tn)	Ratio of correct predictions to all predictions
Error rate	(fp + fn) / (tp + fp + fn + tn)	Ratio of false predictions to all predictions

K-Fold cross validation is a technique used to better evaluate model performance. The data set is split into 10 folds, and for every fold the model is fit on the other 9 folds and makes predictions on the remaining fold. This means the model will be fitted and provide predictions 10 times, with the final result metrics being the average of the individual folds'. Both the individual fold and averaged metrics for all models can be found in the Results section.

Source Code & Annotations

Input Reading & Setup

Dependencies include sklearn for Decision Tree, Random Forest, K-Fold, and metrics and Tensorflow for LSTM. Pyplot and Seaborn are optional and only needed if you wish to display the correlation matrix which by default is disabled.

```
import pandas as pd
#import matplotlib.pyplot as plt
#import seaborn as sns
import sklearn
import warnings
import tensorflow
import numpy as np
from sklearn import metrics as met
from time import process_time
import os.path


from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold

from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM

warnings.filterwarnings('ignore')
```

The data is read from the file ‘default of credit card clients.csv’--ensure that the data file is in the same directory as you are running the program, or it will not be found. If the file is not found or it does not meet the minimum

data threshold of 1 feature column, 1 label column, and 10 rows, the program will exit.

```
#https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients
if os.path.isfile('default of credit card clients.csv'):
    df = pd.read_csv('default of credit card clients.csv')
else:
    print('Input file does not exist')
    input("Press 'Enter' to exit")
    quit()

#df= pd.read_csv('test.csv')
if (df.shape[1] < 2 or df.shape[0] < 10):
    print('Input is not large enough--Ensure there are at least 2 columns and at least 10 rows')
    input("Press 'Enter' to exit")
    quit()

x = df.copy()
y = df.copy()
```

You will be prompted to choose a classification model. You may either enter the number(1, 2, or 3) or the name of the model. If you do not enter a valid model, Random Forest will be chosen as the default.

```
mod = str(input("Choose classification model: \n 1. Random Forest \n 2. Decision Tree \n 3. LSTM \n"))
```

```
Choose classification model:
 1. Random Forest
 2. Decision Tree
 3. LSTM
```

x and y are copies of the initial dataframe. x contains the features while y contains the labels. This is achieved by dropping all but those columns from each dataframe.

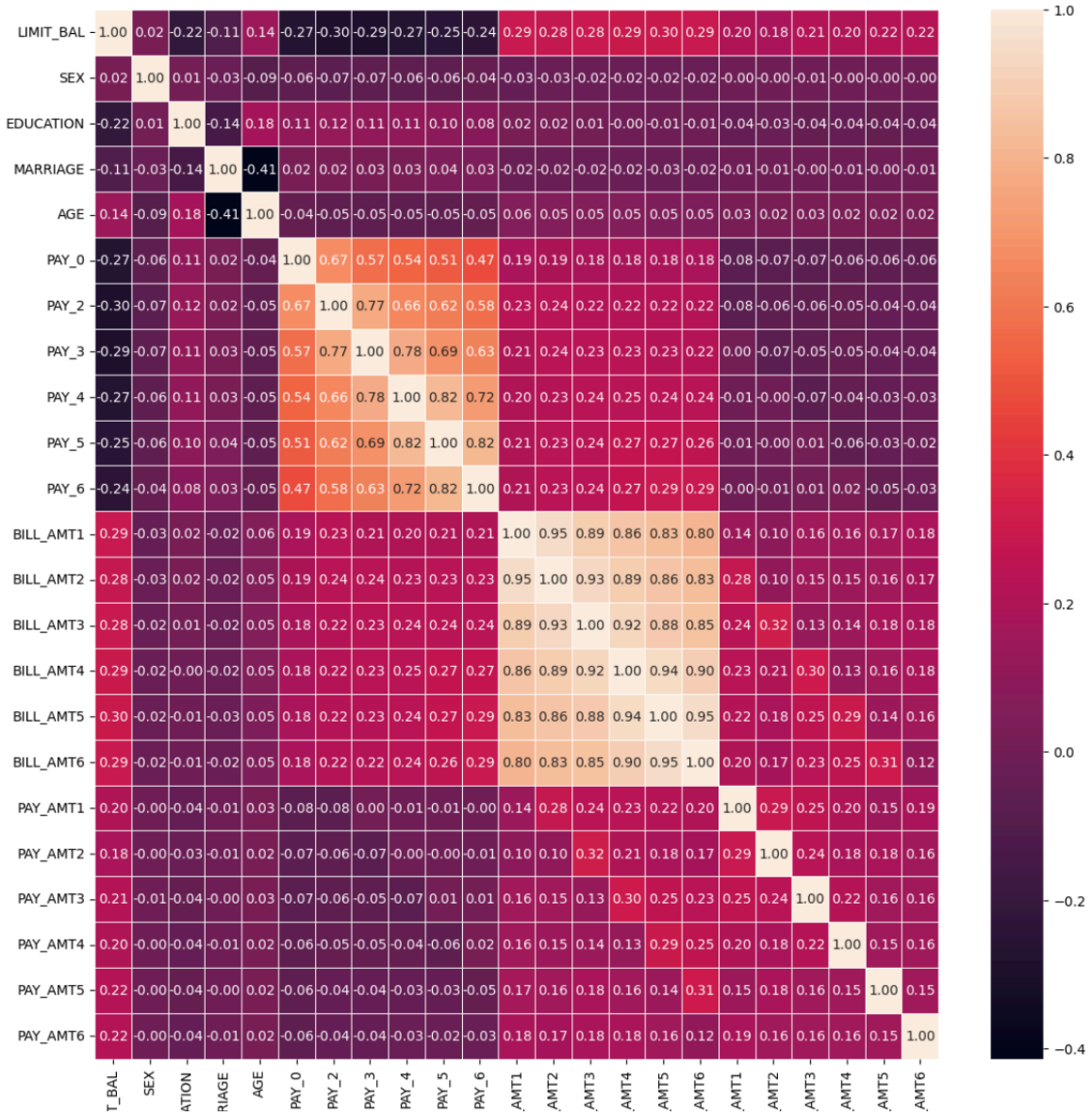
```
#Split data into features and labels
#drop last col
x.drop(x.iloc[:,len(x.columns) - 1: len(x.columns)], inplace=True, axis=1)

#drop all but last col
y.drop(y.iloc[:,0:len(y.columns) - 1], inplace=True, axis=1)
```

The following blocks contain code for creating and displaying a correlation matrix and finding the count and percentage of positive and negative examples in the dataset. These are not necessary for the functionality of the program, but you may enable them by uncommenting the code. Note that you must also uncomment the additional dependencies from the import block and ensure they are available on your system.

```
# Creating a correlation matrix and displaying it using a heatmap
#fig, axis = plt.subplots(figsize=(14, 14))
#correlation_matrix = x.corr()
#sns.heatmap(correlation_matrix, annot=True, linewidths=.5, fmt='.2f', ax=axis)
#plt.show()
```

```
#negative_outcomes, positive_outcomes = y.value_counts()
#total_samples = y.count()
#print('----------Checking for Data Imbalance----------')
#print('Number of Positive Outcomes: ', positive_outcomes)
#print('Percentage of Positive Outcomes: {}%'.format(round((positive_outcomes /
#total_samples) * 100, 2)))
#print('Number of Negative Outcomes : ', negative_outcomes)
#print('Percentage of Negative Outcomes: {}%'.format(round((negative_outcomes /
#total_samples) * 100, 2)))
#print('\n')
```

Below is the output for the correlation matrix block. Most of the attributes have low correlation except for the bill amounts.



Below is the output for the positive/negative split check.

```
----------Checking for Data Imbalance-----------
Number of Positive Outcomes:  6636
Percentage of Positive Outcomes: default payment next month    22.12
dtype: float64%
Number of Negative Outcomes :  23364
Percentage of Negative Outcomes: default payment next month    77.88
```

Model Creation, Training, and Prediction

The amount of K-Fold splits are set to 10 with a constant random state to ensure consistency between executions. This is considered the start time to judge how long each model takes.

```
kf = KFold(n_splits=10, shuffle=True, random_state=42)
stime = process_time()
```

Depending on the earlier input, setup is performed for one of the models. For Random Forest, the number of estimators is set to 30. For LSTM, the tensorflow random seed is also set to a constant value and the sequential model is created, with an LSTM input layer and Dense output layer with the Sigmoid activation function. This will result in a float between 0 and 1, with values closer to 0 implying a stronger negative classification and values closer to one the same for positive classification. As this is a binary classification, Binary Crossentropy is chosen as the loss function.

```
if (mod == '1' or mod == 'Random Forest' or mod == 'random forest'):
    mod = '1'
    model = RandomForestClassifier(n_estimators = 30)
    print("Random Forest\n")
elif (mod == '2' or mod == 'Decision Tree' or mod == 'decision tree'):
    mod = '2'
    model = DecisionTreeClassifier()
    print("Decision Tree\n")
elif (mod == '3' or mod == 'LSTM' or mod == 'lstm'):
    mod = '3'
    print("LSTM\n")
    tensorflow.random.set_seed(42)
    model = Sequential()
    model.add(LSTM(32, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
else:
    mod = '1'
    model = RandomForestClassifier(n_estimators = 30)
    print("Defaulted to Random Forest")
```

Using K-Fold, the data is split 10 ways. Each fold is used as test data with the other 9 folds as training data. The feature data(x_train and x_test) is normalized by subtracting its mean and dividing by its standard deviation, meaning all features will have a mean of 0 and standard deviation of 1. The

model is then fitted to the training data before making predictions on the test data.

If the model chosen is LSTM, the data is reshaped to three dimensional numpy arrays. As the dataset is 2D, a constant 1 is added as the third dimension. The model trains for 15 epochs for each fold before making predictions, where a prediction greater than or equal to 0.5 is positive and otherwise it is negative.

```
resultDF = pd.DataFrame()

for i, (train_index, test_index) in enumerate(kf.split(x), start=1):
    print("Fold " + str(i) + "\n")
    # Splitting the data
    x_train, x_test = x.iloc[train_index], x.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    for dataset in [x_train, x_test, y_train,
        y_test]:
        dataset.reset_index(drop=True, inplace=True)

    x_train = (x_train - x_train.mean()) / x_train.std()
    x_test = (x_test - x_test.mean()) / x_test.std()

    if mod == '3':
        #Reshape data for LSTM
        x_train = x_train.to_numpy()
        x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))
        x_test = x_test.to_numpy()
        x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))
        y_train = y_train.to_numpy()
        y_test = y_test.to_numpy()

        model.fit(x_train, y_train, epochs=15, verbose=0, validation_data=(x_test, y_test))
        y_pred = (model.predict(x_test) >= 0.5).astype("int32")
    else:
        model.fit(x_train, y_train)
        y_pred = model.predict(x_test)
```

Metrics

For each fold, values for TN, FP, TP, and FN are found using sklearn's confusion matrix. These values are used to find the rest of the metrics. All of the fold metrics are added to the result dataframe and printed to the screen.

```
#individual fold metrics
cm = met.confusion_matrix(y_test, y_pred, labels=[0, 1])
tn, fp, fn, tp = cm.ravel()

#recall/TPR
r = tp / (tp + fn)
tnr = tn / (tn + fp)
fpr = fp / (tn + fp)
fnr = fn / (tp + fn)
#precision
p = tp / (tp + fp)
#f1 measure
f1 = 2 * (p * r)/(p + r)
#accuracy
acc = (tp + tn) / (tp + fp + fn + tn)
#error rate
err = (fp + fn) / (tp + fp + fn + tn)

row  = pd.DataFrame({'TP': [tp], 'FP': [fp], 'TN': [tn], 'FN': [fn], 'Recall': [r], 'TNR': [tnr], 'FPR': [fpr],'FNR': [fnr],
                     'Precision': [p],'F1 Measure': [f1],'Accuracy': [acc],'Error Rate': [err]})
resultDF = pd.concat([resultDF, row], ignore_index = True)
print(row)
```

After the final fold, the average metrics will be calculated as the mean of all the fold metrics. This is considered the end of each model's timer.

```
#average metrics
avgResultDF = resultDF.mean(axis=0)
etime = process_time()
```

The results for all folds individually, the average metrics, and the time taken will be written to the screen.

```
print("Results: \n")
print(resultDF)
```

```
Results:

    TP   FP    TN   FN    Recall       TNR       FPR       FNR  Precision  \
0  286  590  1751  373  0.433991  0.747971  0.252029  0.566009   0.326484
1  263  528  1818  391  0.402141  0.774936  0.225064  0.597859   0.332491
2  272  445  1908  375  0.420402  0.810880  0.189120  0.579598   0.379358
3  330  627  1690  353  0.483163  0.729391  0.270609  0.516837   0.344828
4  263  392  1976  369  0.416139  0.834459  0.165541  0.583861   0.401527
5  245  406  1954  395  0.382812  0.827966  0.172034  0.617188   0.376344
6  316  614  1704  366  0.463343  0.735116  0.264884  0.536657   0.339785
7  276  432  1860  432  0.389831  0.811518  0.188482  0.610169   0.389831
8  252  523  1822  403  0.384733  0.776972  0.223028  0.615267   0.325161
9  300  525  1799  376  0.443787  0.774096  0.225904  0.556213   0.363636

   F1 Measure  Accuracy  Error Rate
0    0.372638  0.679000    0.321000
1    0.364014  0.693667    0.306333
2    0.398827  0.726667    0.273333
3    0.402439  0.673333    0.326667
4    0.408702  0.746333    0.253667
5    0.379551  0.733000    0.267000
6    0.392060  0.673333    0.326667
7    0.389831  0.712000    0.288000
8    0.352448  0.691333    0.308667
9    0.399734  0.699667    0.300333
```

```
print("Average Metrics:\n")
print(avgResultDF)
```

```
Average Metrics:

TP              280.300000
FP              508.200000
TN             1828.200000
FN              383.300000
Recall            0.422034
TNR               0.782331
FPR               0.217669
FNR               0.577966
Precision         0.357944
F1 Measure        0.386024
Accuracy          0.702833
Error Rate        0.297167
```

```
print("Time Taken: " + str(etime - stime) + " seconds\n")
```

```
Time Taken: 7.640625 seconds
```

```
print('\n')
input("Press 'Enter' to exit")
```

```


Press 'Enter' to exit
```

Results & Discussion

Below are the individual fold metrics, the average metrics, and the time taken for each model.

Random Forest had a decent 81% accuracy with middling 0.64 precision and 0.44 F1 measure. Decision Tree had a lower accuracy of 70% and low precision of 0.36 and F1 measure of 0.38. It was slightly faster than Random Forest, but not by an amount that would make it worth the performance drop. LSTM took by far the longest out of all 3 models, taking over 47 times as long as Random Forest. It did have slightly better performance at 82% accuracy, 0.67 precision and 0.47 F1 measure, but the extreme length makes it unappealing for such a minor increase. Overall, Random Forest performed the best out of the three models, having nearly the same performance as LSTM at a fraction of the training time.

All models had a much higher accuracy on negative samples than positive samples, likely due to the majority of data samples being part of the negative class. This difference between recall and TNR was the least lopsided for Decision Tree, but the overall accuracy suffered for it. By predicting many more negatives than positives, Random Forest and LSTM had worse recall, but they correctly classified almost all negative samples. This is not a bad outcome; considering that there will always be more customers that do not default compared to those that do, it is better to give them the benefit of the doubt unless they show particular signs of defaulting. However, there is room for improvement.

To improve the models' performance, more positive data samples would be needed. As just mentioned, most customers will not default, so customer data will always be lopsided in favor of the negative class. However, testing some more balanced splits--such as 70-30 or 60-40--instead of 80-20 could improve the ability of the models to distinguish the positive samples without impairing its ability to classify negative examples.

Random Forest

Results:

	TP	FP	TN	FN	Recall	TNR	FPR	FNR	Precision	F1 Measure	Accuracy	Error Rate
0	228	135	2206	431	0.345979	0.942332	0.057668	0.654021	0.628099	0.446184	0.811333	0.188667
1	224	126	2220	430	0.342508	0.946292	0.053708	0.657492	0.640000	0.446215	0.814667	0.185333
2	205	121	2232	442	0.316847	0.948576	0.051424	0.683153	0.628834	0.421377	0.812333	0.187667
3	254	151	2166	429	0.371889	0.934830	0.065170	0.628111	0.627160	0.466912	0.806667	0.193333
4	217	91	2277	415	0.343354	0.961571	0.038429	0.656646	0.704545	0.461702	0.831333	0.168667
5	200	123	2237	440	0.312500	0.947881	0.052119	0.687500	0.619195	0.415369	0.812333	0.187667
6	225	153	2165	457	0.329912	0.933995	0.066005	0.670088	0.595238	0.424528	0.796667	0.203333
7	213	97	2195	495	0.300847	0.957679	0.042321	0.699153	0.687097	0.418468	0.802667	0.197333
8	229	119	2226	426	0.349618	0.949254	0.050746	0.650382	0.658046	0.456630	0.818333	0.181667
9	217	132	2192	459	0.321006	0.943201	0.056799	0.678994	0.621777	0.423415	0.803000	0.197000

Average Metrics:

TP	221.200000
FP	124.800000
TN	2211.600000
FN	442.400000
Recall	0.333446
TNR	0.946561
FPR	0.053439
FNR	0.666554
Precision	0.640999
F1 Measure	0.438080
Accuracy	0.810933
Error Rate	0.189067

Time Taken: 26.015625 seconds

Decision Tree

Results:

	TP	FP	TN	FN	Recall	TNR	FPR	FNR	Precision
0	278	572	1769	381	0.421851	0.755660	0.244340	0.578149	0.327059
1	259	520	1826	395	0.396024	0.778346	0.221654	0.603976	0.332478
2	256	434	1919	391	0.395672	0.815555	0.184445	0.604328	0.371014
3	325	575	1742	358	0.475842	0.751834	0.248166	0.524158	0.361111
4	257	397	1971	375	0.406646	0.832348	0.167652	0.593354	0.392966
5	239	435	1925	401	0.373437	0.815678	0.184322	0.626563	0.354599
6	324	631	1687	358	0.475073	0.727783	0.272217	0.524927	0.339267
7	279	415	1877	429	0.394068	0.818935	0.181065	0.605932	0.402017
8	252	525	1820	403	0.384733	0.776119	0.223881	0.615267	0.324324
9	287	512	1812	389	0.424556	0.779690	0.220310	0.575444	0.359199

	F1 Measure	Accuracy	Error Rate
0	0.368456	0.682333	0.317667
1	0.361479	0.695000	0.305000
2	0.382947	0.725000	0.275000
3	0.410613	0.689000	0.311000
4	0.399689	0.742667	0.257333
5	0.363775	0.721333	0.278667
6	0.395846	0.670333	0.329667
7	0.398003	0.718667	0.281333
8	0.351955	0.690667	0.309333
9	0.389153	0.699667	0.300333

Average Metrics:

TP	275.600000
FP	501.600000
TN	1834.800000
FN	388.000000
Recall	0.414790
TNR	0.785195
FPR	0.214805
FNR	0.585210
Precision	0.356404
F1 Measure	0.382192
Accuracy	0.703467
Error Rate	0.296533

Time Taken: 7.4375 seconds

LSTM

Results:

	TP	FP	TN	FN	Recall	TNR	FPR	FNR	Precision
0	156	89	2252	503	0.236722	0.961982	0.038018	0.763278	0.636735
1	265	161	2185	389	0.405199	0.931373	0.068627	0.594801	0.622066
2	240	132	2221	407	0.370943	0.943901	0.056099	0.629057	0.645161
3	263	125	2192	420	0.385066	0.946051	0.053949	0.614934	0.677835
4	227	79	2289	405	0.359177	0.966639	0.033361	0.640823	0.741830
5	235	136	2224	405	0.367188	0.942373	0.057627	0.632812	0.633423
6	253	83	2235	429	0.370968	0.964193	0.035807	0.629032	0.752976
7	268	111	2181	440	0.378531	0.951571	0.048429	0.621469	0.707124
8	266	157	2188	389	0.406107	0.933049	0.066951	0.593893	0.628842
9	268	147	2177	408	0.396450	0.936747	0.063253	0.603550	0.645783

	F1 Measure	Accuracy	Error Rate
0	0.345133	0.802667	0.197333
1	0.490741	0.816667	0.183333
2	0.471050	0.820333	0.179667
3	0.491130	0.818333	0.181667
4	0.484009	0.838667	0.161333
5	0.464886	0.819667	0.180333
6	0.497053	0.829333	0.170667
7	0.493100	0.816333	0.183667
8	0.493506	0.818000	0.182000
9	0.491292	0.815000	0.185000

Average Metrics:

TP	244.100000
FP	122.000000
TN	2214.400000
FN	419.500000
Recall	0.367635
TNR	0.947788
FPR	0.052212
FNR	0.632365
Precision	0.669177
F1 Measure	0.472190
Accuracy	0.819500
Error Rate	0.180500

Time Taken: 1228.171875 seconds